

Joint 3-class Lung Segmentation — SegResNet, pseudo-LIDC (full corpus w/ pseudo lung labels) — sliding-window (128^3), CE weight 10

1. Task

End-to-end 3-class semantic segmentation of a full chest CT into {background, lung, nodule}. Same task as the paper's `joint_segresnet_pseudo` — the differences are a reduced nodule CE weight (10 instead of 100) and the inference recipe: **sliding-window over 128^3 patches at native resolution** rather than a single forward on a 256^3 resample.

- **Input** (1, H, W, D) CT at native voxel grid.
- **Training input** 128^3 patches sampled via `RandCropByLabelClassesd` with class-balanced ratios `[bg, lung, nodule] = [0.2, 0.3, 0.5]`, `num_samples = 4` per volume.
- **Inference** MONAI `sliding_window_inference`, ROI (128, 128, 128), `overlap = 0.5`, `sw_batch_size = 4`, Gaussian blending, output at input resolution.
- **Output** (3, H, W, D) softmax logits: 0 = bg, 1 = lung, 2 = nodule.

2. Model

Property	Value
Architecture family	SegResNet
Spatial dims	3
Input channels	1
Output channels	3 (softmax)
Init. feature width	16
Trainable parameters	20,663,555

3. Data

Source	Role
NLST	train + val + test
NSCLC-Radiomics	train + val + test
LIDC-IDRI	excluded (no GT lung labels available)

- **Split:** `unified_v2.json` — patient-grouped, dataset-stratified.

- **Preprocessing:** none beyond intensity clipping / normalization; the full CT enters the sliding-window inference at its native voxel grid.

4. Training

Setting	Value
Loss	Multiclass Focal Tversky + weighted CE ($\alpha=0.3$, $\beta=0.7$, $\gamma=2.0$, $\lambda_{CE}=0.1$, class weights [1, 1, 10])
Optimizer	Adam
Learning rate	1×10^{-5}
Weight decay	1×10^{-5}
LR schedule	Cosine annealing, $T_{max} = 400$, $\eta_{min} = 1 \times 10^{-6}$
Batch size	2 volumes $\times$ 4 patches = 8 patches / step
Patch size	128^3 (class-biased random crop, ratios [bg, lung, nodule] = [0.2, 0.3, 0.5])
Epochs	400
Random seed	42
Mixed precision	bf16
Augmentation	3D flips, 90° rotations, elastic rotation, zoom, intensity scale/shift, Gaussian noise/blur, contrast
Hardware	1 $\times$ NVIDIA H100 80 GB (fat01)

Best checkpoint recorded at **epoch 359 / 400**.

5. Evaluation protocol

Evaluated on the `unified_v2` **test** split (325 series). All voxel-level metrics micro-averaged over the whole split. Per-class metrics computed against argmax predictions.

6. Results

Ticket-required metrics (3-class average)

Metric	Value
mean IoU (3-cls)	0.7617
mean IoU (lung + nodule)	0.6451
Accuracy	0.995278
Macro Precision (lung + nodule)	0.6657

Metric	Value
Macro Recall (lung + nodule)	0.9187

Per-class breakdown

Class	Dice (micro)	Recall	Precision	IoU
background	0.9975	0.9971	0.9978	0.9949
lung	0.9769	0.9772	0.9766	0.9548
nodule	0.5024	0.8603	0.3547	0.3354

Per-case Dice distribution — nodule class (325 test cases)

Statistic	Value
Mean	0.3873
Std	0.2554
Min	0.0000
p25	0.1717
Median	0.3735
p75	0.5898
Max	0.9294

Voxels evaluated: 17,795,943,918. Cases: 325.

7. Notes on interpretation

Lung Dice at ≈ 0.98 is essentially saturated across all architectures and inference recipes — the lung is a large, well-defined foreground class. Nodule Dice is the real signal.

Sliding-window inference at 128^3 trades global lung context for preserved local resolution. On this corpus the trade tends to be neutral-to-negative on nodule Dice compared to the 256^3 resize baseline — patches see only $\sim 15\%$ of a typical lung volume, which limits anatomy-conditional reasoning.

Instance-level metrics (per-nodule)

Connected-component analysis with 26-connectivity, pooled over the held-out test split, on the native voxel grid.

- **Instance recall** — GT nodules touched by at least one predicted voxel, over all GT nodules.
- **Instance precision** — predicted components touching at least one GT voxel, over all predicted components.

Metric	Value
GT nodules (components)	947
GT nodules hit	846
Instance recall	0.8933
Predicted components	13,456
Predicted components hitting GT	853
Instance precision	0.0634

No minimum-size filtering is applied — every predicted component counts, so single-voxel false positives lower instance precision. Produced by `scripts/instance_metrics.py --config configs/joint_segresnet_pseudo_sw_ce10.yaml --ckpt <ckpt> --split test` (JSON: `results_instance/joint_segresnet_pseudo_sw_ce10.json`).

8. Reproducibility

- Config `/home/werner/nodule-segmentation/configs/joint_segresnet_pseudo_sw_ce10.yaml`
- Checkpoint `stage3_joint/checkpoints_segresnet_pseudo_sw_ce10/best_model.pth`
- Metrics JSON `/tmp/claude-1002/-home-werner-nodule-segmentation/c2faa0a0-5a75-48f5-8288-377df40092b7/scratchpad/ce10_test/joint_segresnet_pseudo.json`
- Command `python stage3_joint/eval_metrics.py --config /home/werner/nodule-segmentation/configs/joint_segresnet_pseudo_sw_ce10.yaml --checkpoint stage3_joint/checkpoints_segresnet_pseudo_sw_ce10/best_model.pth --split test`